

Student Name: Atharva Mohite

Student Session: cs6220

CS 6220 Big Data Systems, Fall 2026 · Homework 2, programming option

CS 6220 BIG DATA SYSTEMS · HOMEWORK 2 · PROGRAMMING OPTION

Five Word2Vec Models, One Classifier

Two models trained from scratch on movie reviews, one downloaded, one fine-tuned from the downloaded weights, and one where the training algorithm itself was reimplemented. Scored on sentiment, on lexical similarity, and on what they cost to run.

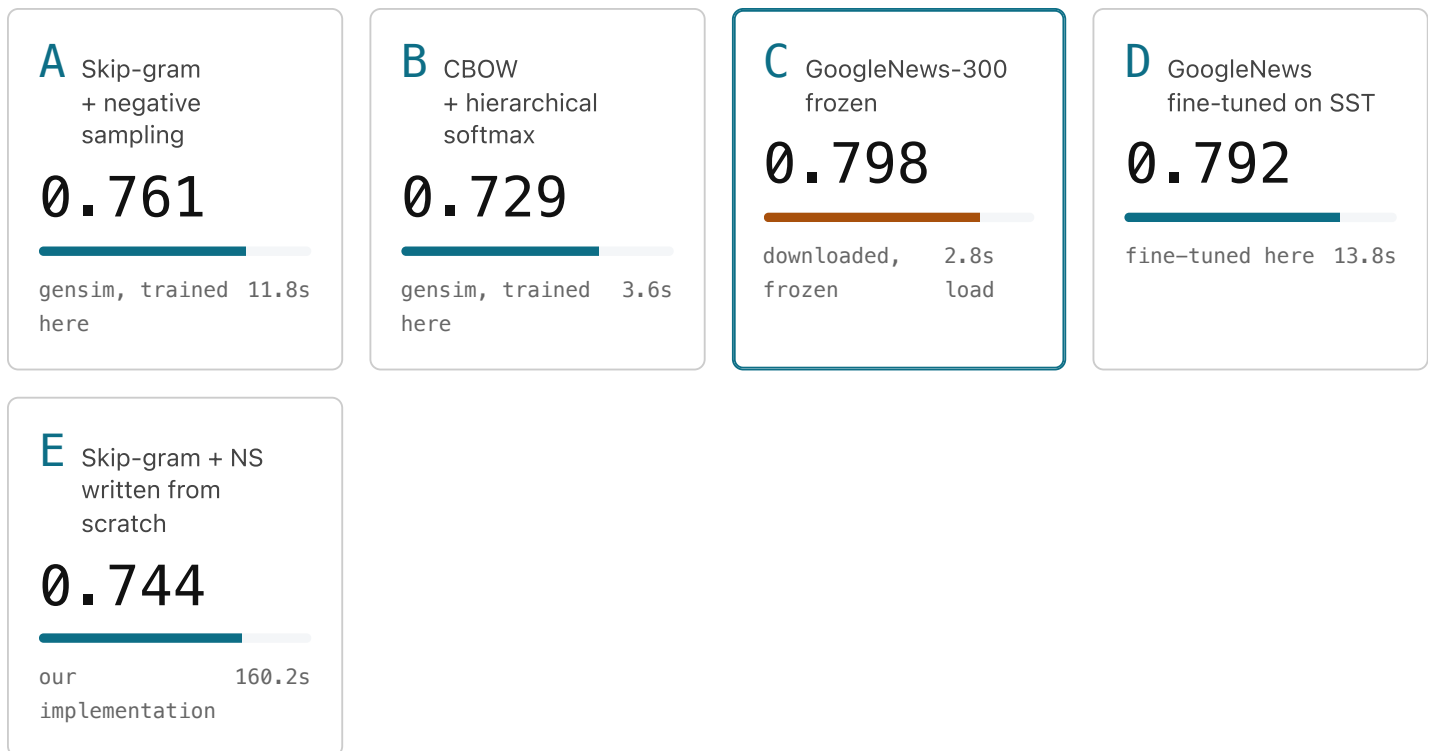
Corpus SST-2 phrases, 633,720 tokens Vocabulary 14,309 Dimensions 300 Test split 1,821 sentences

Machine M2 Air

WHERE THINGS STAND

Five models, one classifier

Every model is scored the same way. Look up a vector per word, average them into a sentence vector, fit logistic regression, test on the same held-out SST-2 sentences. The classifier never changes and is never tuned per model, so any gap between these numbers belongs to the embeddings and to nothing else.



Isolating the warm start needed a control run. A against D is the obvious comparison, but it is not clean: D also uses a lower learning rate (0.005 against 0.025) and fewer epochs (8 against 10), so that gap mixes initialisation with the optimiser schedule. **A2** is A re-run with D's exact schedule and random initialisation, which leaves the starting point as the only difference. A2 scores 0.664 against D's 0.792, so the warm start is worth **+0.127**, not the +0.031 the A-versus-D gap suggests. The looser comparison understated it, because A's more aggressive schedule was partly compensating for its random start.

MODEL	ARCHITECTURE	LOSS	TRAINED ON	ACCURACY	F1	OOV
A	Skip-gram	Negative sampling, k=10	633,720 tok	0.7611	0.7757	0.066
B	CBOW	Hierarchical softmax	633,720 tok	0.7293	0.7501	0.066
C	Skip-gram	Negative sampling, k=5	100,000,000,000 tok	0.7979	0.8107	0.271
D	Skip-gram	Negative sampling, k=10	633,720 tok	0.7919	0.8089	0.066
E	Skip-gram	Negative sampling, k=10	633,720 tok	0.7441	0.7610	0.066
A2	Skip-gram	Negative sampling, k=10	633,720 tok	0.6645	0.7070	0.066

A2 is not a sixth model so much as a control. It exists only to make the A-versus-D claim above testable, and it is reported here rather than hidden because its score (0.664) is the lowest in the table and that is the point.

DELIVERABLE 3

Five test examples, ranked by every model

The sentiment task on held-out test data, which is what the SST corpus is here for. Each example carries two ranked outputs per model: the classifier's ranked labels with probabilities, and the top-5 nearest *training* sentences by cosine in that model's embedding space. Neighbour agreement is the share of those retrieved training sentences whose gold label matches the test sentence, so it says whether the embedding placed the test sentence among the right evidence.

The five were chosen by rule rather than by eye, and the rule that selected each is named above it. Scores across the five: A 3/5, B 3/5, C 4/5, D 3/5, E 3/5.

EXAMPLE 1 · PLAIN POSITIVE

this is one of polanski 's best films .

gold label: **positive**

MODEL	RANK 1	RANK 2	RESULT	NEIGHBOUR AGREEMENT
A	positive 0.838	negative 0.162	correct	1.0
B	positive 0.694	negative 0.306	correct	1.0
C	positive 0.855	negative 0.145	correct	1.0
D	positive 0.808	negative 0.192	correct	1.0
E	positive 0.656	negative 0.344	correct	1.0

D retrieves

this is one of the year 's best films . · 0.99

is one of the year 's best films . · 0.98

is one of the year 's best films · 0.97

Retrieved training sentences, nearest first, with cosine. Highlighted where the training sentence's own label matches this test sentence's gold label.

EXAMPLE 2 · PLAIN NEGATIVE

arnold 's jump from little screen to big will leave frowns on more than a few faces .

gold label: **negative**

MODEL	RANK 1	RANK 2	RESULT	NEIGHBOUR AGREEMENT
A	positive 0.593	negative 0.407	wrong	1.0
B	negative 0.534	positive 0.466	correct	1.0
C	negative 0.668	positive 0.332	correct	0.0
D	positive 0.512	negative 0.488	wrong	1.0
E	negative 0.546	positive 0.454	correct	0.8

D retrieves

big fat liar is little more than home alone raised to a new , self-deprecating l · 0.93

is little more than home alone raised to a new , self-deprecating level . · 0.92

the film runs on a little longer than it needs to · 0.92

Retrieved training sentences, nearest first, with cosine. Highlighted where the training sentence's own label matches this test sentence's gold label.

EXAMPLE 3 · POSITIVE DESPITE NEGATION

finally , a genre movie that delivers -- in a couple of genres , no less .

gold label: **positive**

MODEL	RANK 1	RANK 2	RESULT	NEIGHBOUR AGREEMENT
A	negative 0.596	positive 0.404	wrong	0.4
B	negative 0.643	positive 0.357	wrong	0.2
C	positive 0.680	negative 0.320	correct	0.4
D	negative 0.508	positive 0.492	wrong	0.8
E	negative 0.597	positive 0.403	wrong	0.4

D retrieves

neither quite a comedy nor a romance , more of an impish divertissement of theme · 0.95

singer/composer bryan adams contributes a slew of songs -- a few potential hits · 0.94

singer/composer bryan adams contributes a slew of songs -- a few potential hits · 0.94

Retrieved training sentences, nearest first, with cosine. Highlighted where the training sentence's own label matches this test sentence's gold label.

EXAMPLE 4 · CONTRASTIVE CLAUSE

but what saves lives on the freeway does not necessarily make for persuasive viewing .

gold label: **negative**

MODEL	RANK 1	RANK 2	RESULT	NEIGHBOUR AGREEMENT
A	negative 0.592	positive 0.408	correct	1.0
B	positive 0.574	negative 0.426	wrong	0.8
C	positive 0.565	negative 0.435	wrong	0.8
D	negative 0.502	positive 0.498	correct	0.8
E	positive 0.614	negative 0.386	wrong	0.6

D retrieves

does n't show enough of the creative process or even of what was created for the · 0.92

if swimfan does catch on , it may be because teens are looking for something to · 0.92

what might have been acceptable on the printed page of iles ' book does not tran · 0.92

Retrieved training sentences, nearest first, with cosine. Highlighted where the training sentence's own label matches this test sentence's gold label.

EXAMPLE 5 · NEGATIVE WITH STACKED NEGATION

no movement , no yuks , not much of anything .

gold label: **negative**

MODEL	RANK 1	RANK 2	RESULT	NEIGHBOUR AGREEMENT
A	negative 0.972	positive 0.028	correct	1.0
B	negative 0.955	positive 0.045	correct	1.0
C	negative 0.964	positive 0.036	correct	1.0
D	negative 0.989	positive 0.011	correct	1.0
E	negative 0.860	positive 0.140	correct	0.6

D retrieves

... no charm , no laughs , no fun , no reason to watch . · 0.92

no charm , no laughs , no fun , no reason to watch . · 0.92

no charm , no laughs , no fun , no reason to watch · 0.91

Retrieved training sentences, nearest first, with cosine. Highlighted where the training sentence's own label matches this test sentence's gold label.

Example 3 separates the pretrained model from everything trained here. `no less` is an intensifier, not a negation, and only GoogleNews reads it that way. Every SST-trained model including the fine-tune takes it as negative.

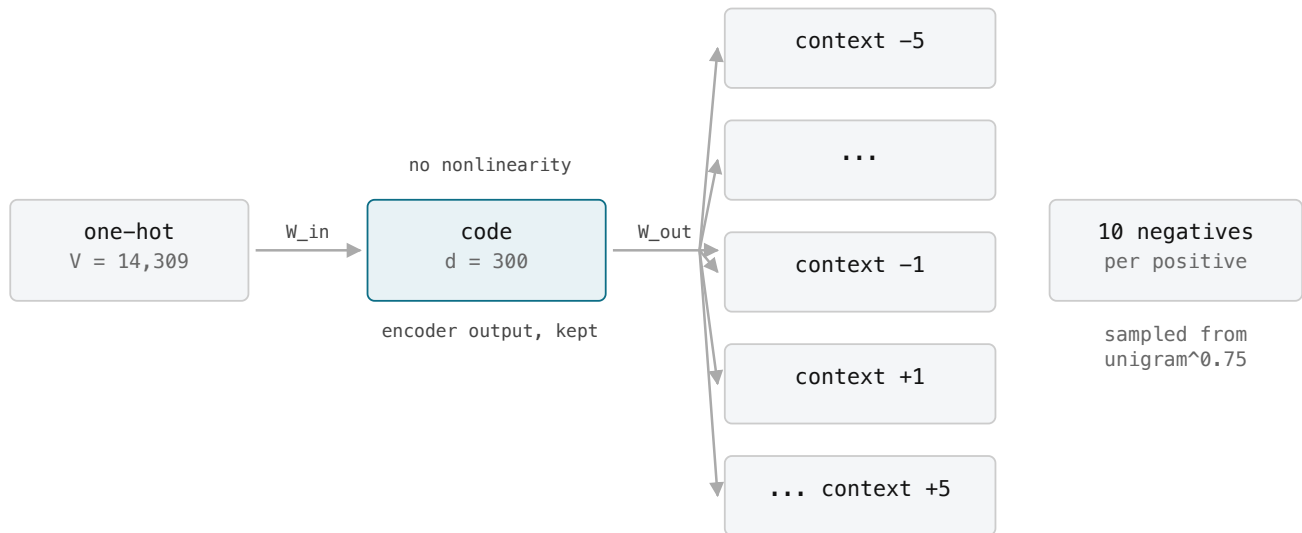
Example 5 is the one to be suspicious of. Every model gets it right with high confidence, and retrieval shows why: the nearest training sentence is `no charm , no laughs , no fun`, a near-duplicate of the test sentence's stacked-negation pattern. The models are right for lexical reasons, not because they composed the negation. That is the same mechanism that makes Example 3 fail, and it happens to point the right way here.

DELIVERABLE 1

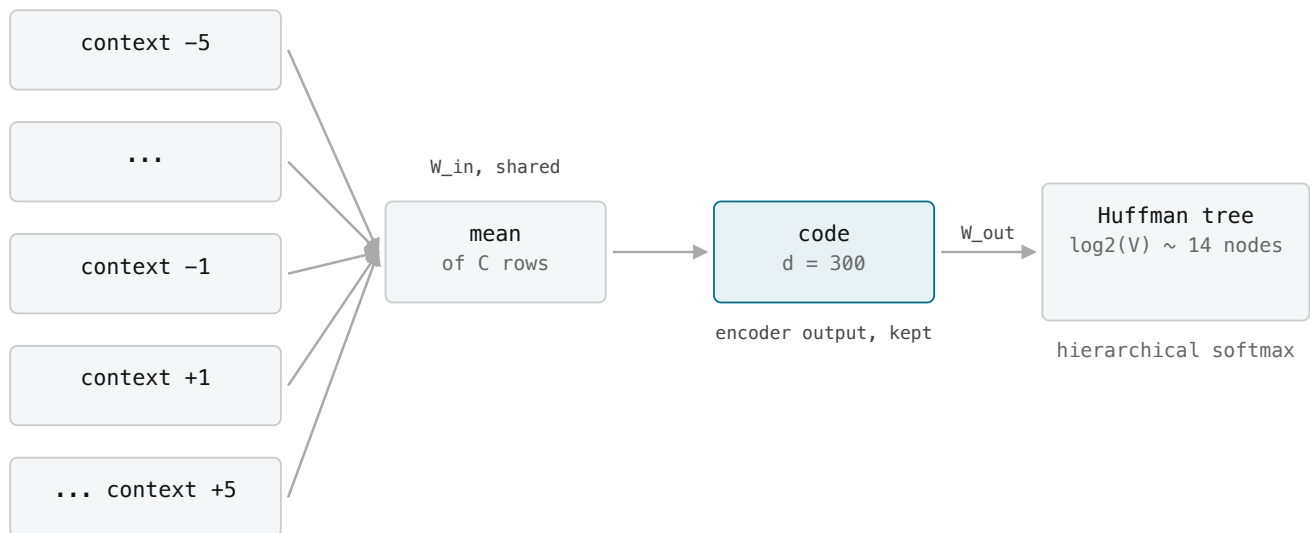
What the two architectures actually are

A word2vec model is two matrices and a dot product. There is no hidden layer and no nonlinearity. `W_in` holds one row per word and is the thing you keep. `W_out` exists only to

compute the training loss and is discarded when training ends.



A and E, skip-gram with negative sampling. One centre word predicts each surrounding word independently. Instead of a softmax over all 14,309 words, each positive pair is scored against 10 negatives drawn from the unigram distribution raised to 0.75.



B, CBOW with hierarchical softmax. The context rows are averaged into a single vector that predicts the centre word. The output is a binary tree over the vocabulary, so one prediction costs $\log_2(V)$ decisions rather than V .

On the handout's encoder / code / decoder framing. It is borrowed from autoencoders and fits word2vec only loosely. An autoencoder reconstructs its own input; word2vec reconstructs a *neighbouring* word, and it throws the decoder away. The mapping used above is the honest one: the one-hot input is the input layer, `W_in` is the encoder, the 300-dimensional row is the code, `W_out` is the decoder, and the context prediction is the output.

DELIVERABLE 2

Training and test data

Sentiment data is SST-2. Two releases exist and the choice matters. The GLUE copy sets every test label to -1 because that split is held out for the leaderboard, so test labels come from the SetFit release instead, and `src/data.py` raises if a loaded test split is not two-class.

SPLIT	SOURCE	ROWS	TOKENS	POSITIVE	USED FOR
train	stanfordnlp/sst2 (phrase level)	67,348	633,720	55.8%	embedding training and classifier fitting
dev	SetFit/sst2	872	17,046	50.9%	held out, unused
test	SetFit/sst2	1,821	35,023	49.9%	every number on this page

Leakage check. The phrase rows are labelled subtrees of the treebank and come from the train split only, but they were still checked against the evaluation splits by exact string match: zero overlap with dev, and one row overlapping test, the fragment `what 's next ?`. It is dropped, which is why 67,349 rows become 67,348.

The pretrained model is GoogleNews-300, genuine skip-gram with negative sampling trained on roughly 100 billion tokens of news text, from code.google.com/archive/p/word2vec/, loaded through gensim-data and capped at the 500,000 most frequent words. GloVe was deliberately not used as the pretrained baseline: it factorises a co-occurrence matrix and is not a Word2Vec model, so it would answer a different question than the one asked.

PRETRAINED MODEL, AS DELIVERABLE 2A ASKS

FIELD	VALUE
Model	GoogleNews-vectors-negative300
Reference	Mikolov, Sutskever, Chen, Corrado and Dean (2013), <i>Distributed Representations of Words and Phrases and their Compositionality</i> , NIPS. arxiv.org/abs/1310.4546
Distribution URL	code.google.com/archive/p/word2vec/ , fetched through gensim-data as <code>word2vec-google-news-300</code>
Training corpus	Google News, roughly 100 billion tokens. The corpus is proprietary and was never released, so it cannot be linked.
Architecture	Skip-gram with negative sampling, 300 dimensions, 3M words and phrases
Test set used here	Google analogy set (<code>questions-words.txt</code>), 19,544 questions, shipped in <code>data/eval/</code>
Accuracy, measured here	0.7561 over 15,434 attempted questions (79.0% coverage, candidate pool restricted to the top 50,000 words)
Accuracy, as published	Not quoted. The Google Code archive page is a JavaScript application and serves no figures to a fetch, and the linked papers report results for models trained in those papers rather than for this released file. Rather than cite a number that could not be verified, the measured figure above is given with its exact conditions.

Representative examples

Deliverable 2b asks for five training examples and four test examples, two positive and two negative. Selected by rule rather than by eye, with the selecting rule named beside each.

FIVE TRAINING EXAMPLES

SELECTED FOR	TEXT	GOLD LABEL
positive, plain	that loves its characters and communicates something rather beautiful about human nature	positive
negative, plain	remains utterly satisfied to remain the same throughout	negative
negation	for those moviegoers who complain that ` they do n't make movies like they used to anymore	negative
contrastive clause	the plot is nothing but boilerplate clichés from start to finish ,	negative
film-domain vocabulary	, good action , good acting , good dialogue , good pace , good cinematography .	positive

FOUR TEST EXAMPLES

SELECTED FOR	TEXT	GOLD LABEL
positive	this is one of polanski 's best films .	positive
positive	finally , a genre movie that delivers -- in a couple of genres , no less .	positive
negative	arnold 's jump from little screen to big will leave frowns on more than a few faces .	negative
negative	no movement , no yuks , not much of anything .	negative

DELIVERABLE 5

Hyperparameters

A and B differ on two axes at once, architecture and loss, which is deliberate: it gives the comparison two things to say rather than one. Everything else is held constant between them.

	A	B	C	D	E
Architecture (sg)	1	0	1	1	1
Hierarchical softmax	0	1	0	0	0
Negatives	10	0	5	10	10
Dimensions	300	300	300	300	300
Window	5	5	5	5	5
Min count	2	2	5	2	2
Epochs	10	10	None	8	10
Initial LR	0.025	0.025	0.025	0.005	0.001
Subsample threshold	0.001	0.001	1e-05	0.001	0.001

DELIVERABLES 3 AND 5

Top-ranked neighbours for five queries

Five words chosen to separate the models rather than flatter them: a polysemous noun, a sentiment adjective, a word whose sense depends on domain, a rare domain term, and the most frequent word in English. Highlighted chips on row D are neighbours that are not in C's list for the same query, so they are what fine-tuning introduced.

plot

MODEL	TOP-1	TOP-2 TO 5	TOP-6 TO 10
A	twists 0.48	pawn 0.46 device 0.46 court 0.46 oh-so 0.45	nifty 0.45 cliché-riddled 0.44 stop- and-start 0.44 triteness 0.44 convenient 0.44
B	ending 0.32	accomplishment 0.28 sweetness 0.26 flourish 0.25 despair 0.25	flashy 0.25 mastering 0.25 clancy 0.25 compressed 0.24 sure-fire 0.24
C	plots 0.76	Plot 0.65 plotting 0.63 plotted 0.60 plotline 0.59	plotters 0.54 storyline 0.53 conspiracies 0.52 conspiracy 0.51 foiled_plot 0.50
D	plots 0.77	plotting 0.68 plotted 0.62 plotline 0.56 storyline 0.52	conspiracies 0.51 conspiracy 0.51 oh- so 0.50 conspirators 0.48 subplots 0.47
E	twists 0.86	device 0.84 oh-so 0.84 halfwit 0.83 court 0.83	stop-and-start 0.81 maneuvers 0.81 sillified 0.81 holes 0.80 nurses 0.80

terrible

MODEL	TOP-1	TOP-2 TO 5	TOP-6 TO 10
A	fitting 0.70	synergistic 0.67 stultifyingly 0.67 reconstruction 0.66 bloodwork 0.66	besotted 0.59 impulse 0.59 propelled 0.58 freaky 0.58 regard 0.57
B	thrilling 0.40	chump 0.36 ballistic 0.34 motion 0.34 paint-by- numbers 0.33	view 0.33 derrida 0.33 ecks 0.33 cinderella 0.33 raises 0.33
C	horrible 0.92	horrendous 0.85 dreadful 0.80 awful 0.75 horrid 0.72	atrocious 0.69 horrific 0.68 bad 0.68 appalling 0.68 horrible_horrible 0.67
D	horrible 0.89	dreadful 0.80 awful 0.78 atrocious 0.72 horrid 0.71	appalling 0.70 horrific 0.68 lousy 0.67 hideous 0.67 clockstoppers 0.67
E	+ 0.85	besotted 0.84 allegedly 0.83 unfulfilling 0.83 dungeons 0.83	phenomenal 0.82 lanes 0.82 retread 0.82 inspired 0.81 snl 0.81

MODEL	TOP-1	TOP-2 TO 5	TOP-6 TO 10
A	raving 0.59	trek 0.59 high-powered 0.58 pop-induced 0.57 jake 0.57	bestowed 0.57 anteing 0.56 wattage 0.56 lone 0.56 indomitability 0.55
B	winners 0.33	kevin 0.31 splendid 0.30 has 0.30 acting 0.30	pulls 0.30 watstein 0.29 contributes 0.29 dustin 0.28 teaming 0.28
C	stars 0.78	superstar 0.73 starlet 0.64 megastar 0.62 heart_throb 0.57	heartthrob 0.54 superstars 0.53 standout 0.52 stardom 0.52 hunk 0.51
D	superstar 0.75	stars 0.74 gyllenhaal 0.60 top-billed 0.60 pop- induced 0.58	first-rate 0.58 standout 0.58 starring 0.56 huston 0.56 legend 0.55
E	jake 0.88	rising 0.87 gyllenhaal 0.85 charismatic 0.85 top- billed 0.82	pop-induced 0.77 willis 0.74 bestowed 0.73 high-powered 0.73 kevin 0.73

cinematography

MODEL	TOP-1	TOP-2 TO 5	TOP-6 TO 10
A	rinzler0.71	water-born 0.70shimmering0.70hennings 0.69lisa 0.69	l. 0.69conrad 0.68italicized0.68robotically 0.68pitted0.67
B	color0.38	gantz 0.38beaches 0.37mattei0.37hotels 0.36	improbability 0.36acting0.35lumpen 0.35procedural0.35reggio 0.34
C	camerawork0.77	cinematographer0.75Cinematography0.70Cinematographer0.67cinematography_editing 0.63	cinematographers0.63Best_Cinematography0.62visual_effects0.60filmmaking 0.60visuals0.59
D	camerawork0.76	cinematographer 0.75visuals0.61screenplay 0.59photography0.59	choreography 0.58filmmaking0.56helmer 0.55lensing0.54narration 0.54
E	hennings0.90	water-born 0.88dubbed0.88naturalistic 0.87murky0.87	shimmering 0.86phenomenal0.86robotically 0.86atrocious0.84acute 0.84

the

MODEL	TOP-1	TOP-2 TO 5	TOP-6 TO 10
A	of 0.40	deleting 0.36 outward 0.35 tully 0.35 1987 0.35	unambitious 0.35 screenplays 0.35 hubristic 0.35 homeric 0.35 indulges 0.35
B	our 0.22	his 0.21 current 0.19 dong 0.16 whose 0.15	german 0.15 of 0.15 my 0.15 in 0.14 art 0.14
C	this 0.59	in 0.54 that 0.53 ofthe 0.52 another 0.47	however 0.47 one 0.47 entire 0.46 its 0.46 which 0.46
D	of 0.63	. 0.59 2002 0.58 spielberg 0.54 entire 0.54	salton 0.53 self-conscious 0.53 pacino 0.53 revenge-of-the-nerds 0.52 auteuil 0.52
E	inside- show-biz 0.94	producing 0.93 pierce 0.93 silent 0.93 1790 0.93	savaged 0.92 democratic 0.92 shoulder 0.92 worlds 0.92 indulges 0.92

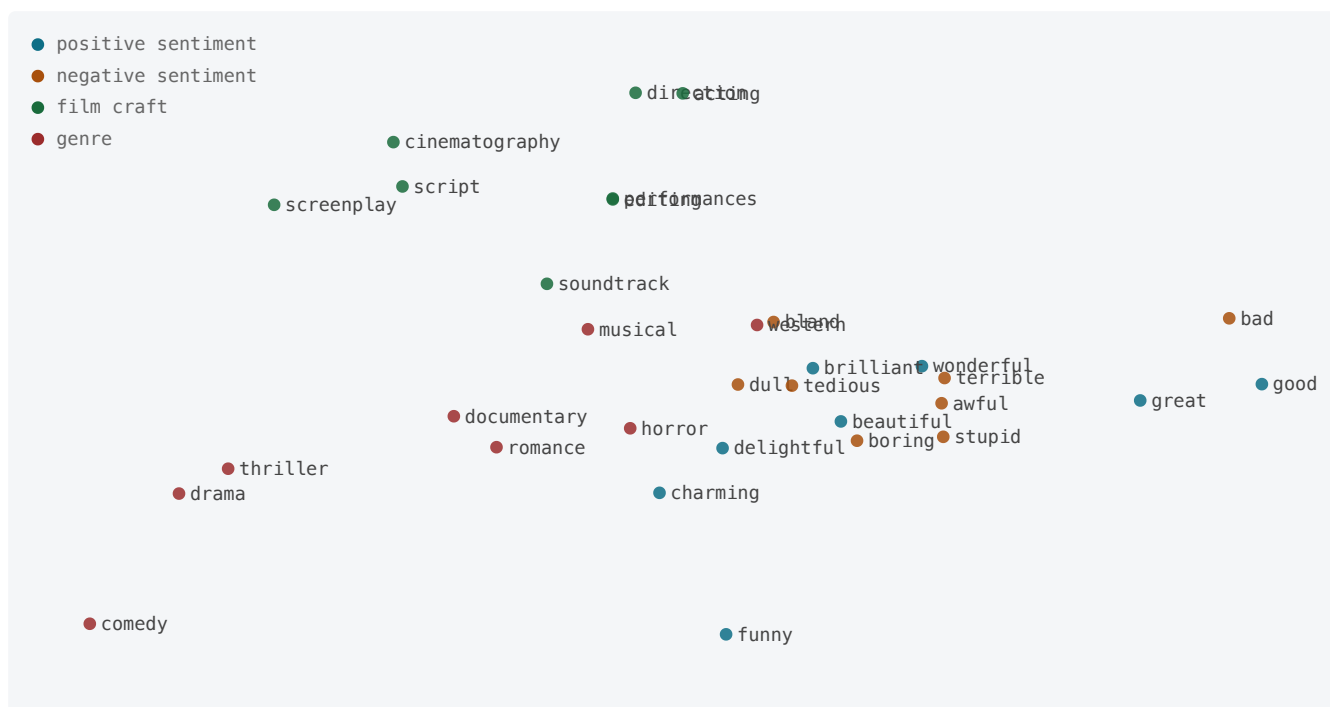
Read the C and D rows against A, B and E. C returns clean synonyms and inflections. D keeps them and adds film-specific associations: `visuals`, `screenplay` and `photography` enter `cinematography`, and `storyline` enters `plot`. A, B and E return something close to noise for every query. At 633,720 tokens there is not enough co-occurrence evidence to place a word, and the next section puts a number on exactly how little.

HANDOUT REQUIREMENT

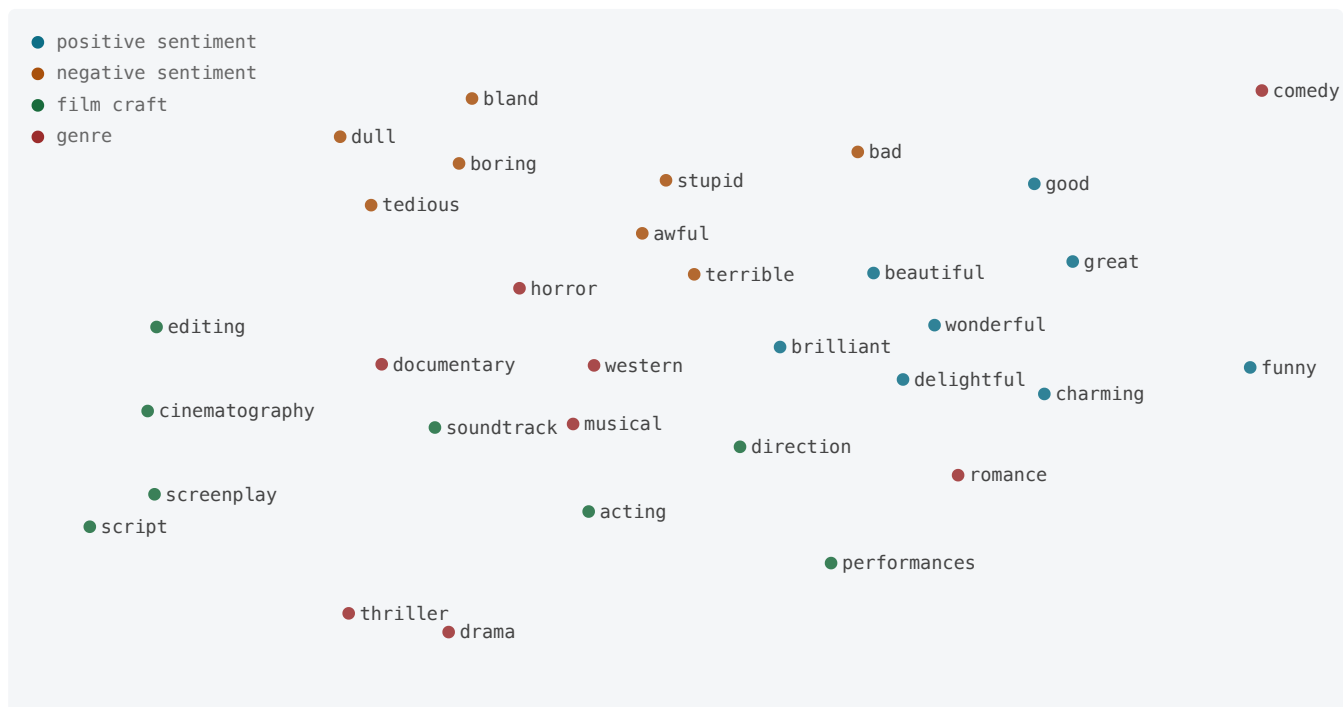
The vector space in two dimensions

The words plotted are fixed in advance and belong to four semantic groups: positive sentiment, negative sentiment, film craft, and genre. Fixing them first means the figure can be judged rather than admired. If the embedding carries semantic structure, group members should land near each other after projection.

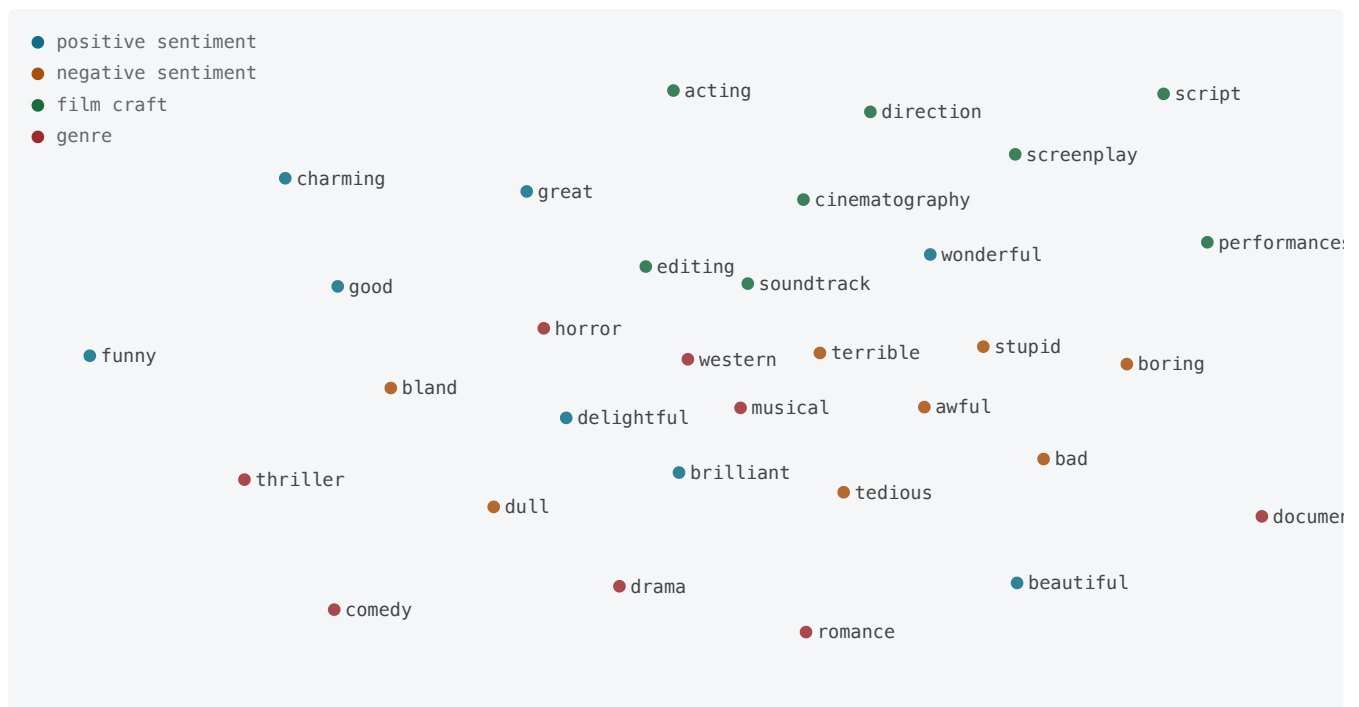
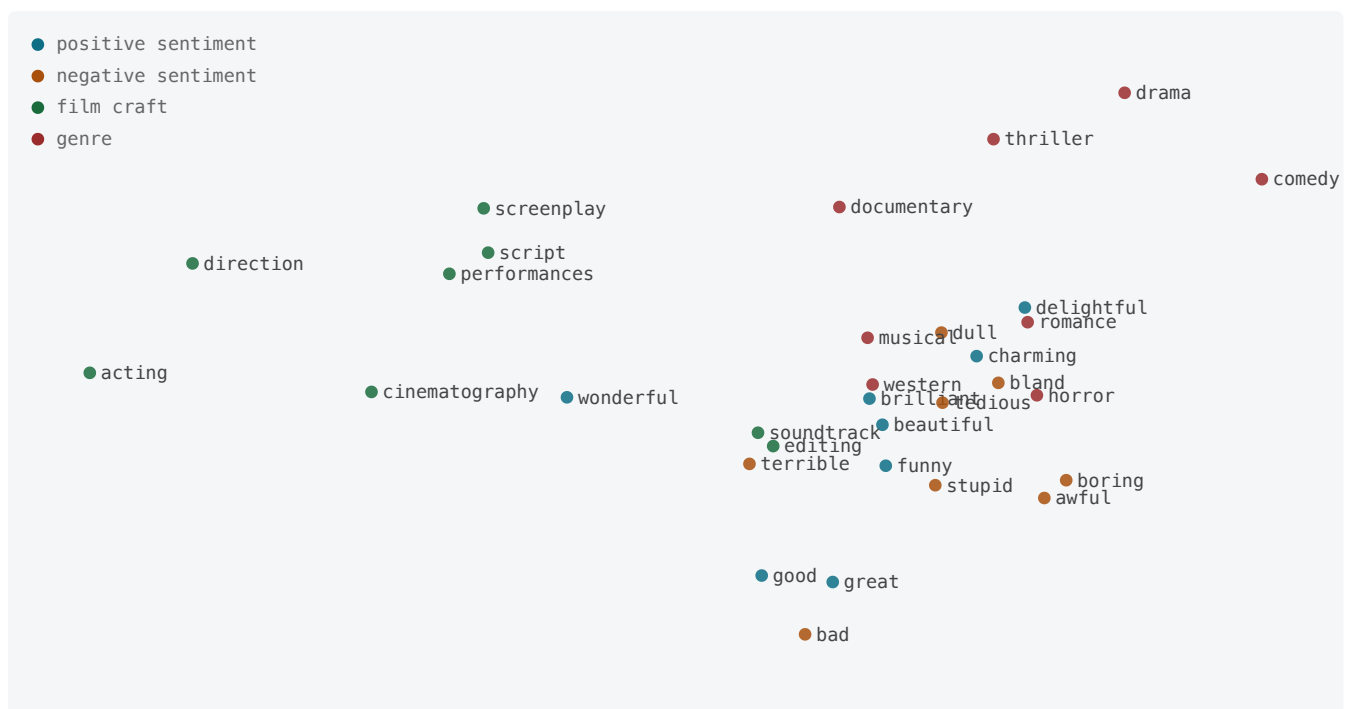
Judging by eye is unreliable, so each figure also reports the silhouette score of the two-dimensional layout against the group labels. Around 0 means the groups are indistinguishable; higher means they separate.



PCA, Model D. 32 words from four fixed semantic groups, explained variance 16.2% and 9.6%. Silhouette of the two-dimensional layout against the group labels: **0.164**.



t-SNE, Model D. 32 words from four fixed semantic groups, perplexity 10. Silhouette of the two-dimensional layout against the group labels: **0.187**.



MODEL	PCA SILHOUETTE	T-SNE SILHOUETTE	WORDSIM-353	ANALOGY ACCURACY
A	+0.036	+0.051	+0.029	0.001
B	+0.035	-0.033	-0.036	0.002
C	+0.120	+0.121	+0.700	0.756
D	+0.164	+0.187	+0.585	0.675
E	-0.004	+0.110	+0.005	0.002

The projections agree with every other measurement. D and C separate the four groups (+0.164 and +0.120 under PCA); A, B and E do not (+0.036, +0.035, -0.004). The same ordering appears in WordSim-353 and in analogy accuracy, which is worth stating because a scatter plot on its own is the easiest figure in this report to over-read. Note also how little variance the first two PCA components capture, under 20% for every model, so the picture is a thin slice of a 300-dimensional space.

HANDOUT REQUIREMENT

Vector arithmetic and analogies

The handout names `king - man + woman = queen` specifically. Below is that query and six others, each showing the ranked answers the model returns, followed by the standard Google analogy benchmark scored per section.

MODEL	QUERY	RANKED ANSWERS
A	king - man + woman	scorpion 0.54 lion 0.50 lear 0.50 nuts 0.49
A	paris - france + italy	beckons 0.72 1995 0.70 marvin 0.70 holland 0.69
A	worst - bad + good	moronic 0.36 drowns 0.36 resources 0.36 bloated 0.35
B	king - man + woman	sugar 0.39 abbot 0.36 ernest 0.36 wildcard 0.35
B	paris - france + italy	1995 0.59 dog-tag 0.53 junk-calorie 0.52 gobble 0.51
B	worst - bad + good	parents 0.31 teen-driven 0.28 made 0.28 shining 0.28
C	king - man + woman	queen 0.71 monarch 0.62 princess 0.59 crown_prince 0.55
C	paris - france + italy	madrid 0.48 rome 0.46 barcelona 0.46 spain 0.46
C	worst - bad + good	best 0.69 strongest 0.63 greatest 0.57 weakest 0.57
D	king - man + woman	queen 0.70 princess 0.60 prince 0.57 queens 0.56
D	paris - france + italy	spain 0.48 diego 0.48 travis 0.47 samuel 0.47
D	worst - bad + good	best 0.58 strongest 0.57 finest 0.56 highest 0.56
E	king - man + woman	middle-aged 0.71 lion 0.71 creek 0.70 haynes 0.70
E	paris - france + italy	insulted 0.92 startled 0.91 stardom 0.91 forewarned 0.91
E	worst - bad + good	moronic 0.69 screenplays 0.69 simmer 0.69 sporting 0.66

Only C and D solve the canonical analogy. Both return `queen` at rank 1 for `king - man + woman`. A, B and E return unrelated words, which is the same conclusion the similarity benchmarks and the projections reach: 633,720 tokens of film review does not build a space where vector arithmetic means anything.

MODEL	ANALOGY ACCURACY	QUESTIONS ATTEMPTED	COVERAGE	CANDIDATE POOL
A	0.0006	3,563 of 19,544	18.2%	top 50,000
B	0.0020	3,563 of 19,544	18.2%	top 50,000
C	0.7561	15,434 of 19,544	79.0%	top 50,000
D	0.6750	3,563 of 19,544	18.2%	top 50,000
E	0.0022	3,563 of 19,544	18.2%	top 50,000

Coverage matters more than accuracy here. gensim drops any question containing an out-of-vocabulary word, so the SST-trained models are scored on 18.2% of the set against C's 79.0%. D's 0.675 is structure inherited from the warm start, not learned from SST.

SUPPORTING ANALYSIS, BEYOND THE REQUIREMENT

Good at sentiment, useless at similarity

The sections above are the required sentiment task. This one is extra, and it exists because the sentiment numbers alone give a misleading picture of what these embeddings know.

Spearman correlation between human similarity ratings and cosine similarity, on three standard benchmarks. Coverage is the share of pairs where both words were in vocabulary, and it is part of the result rather than a footnote: the SST-trained models are scored on roughly half of WordSim-353 because film reviews never mention the rest.

MODEL	WORDSIM-353	SIMLEX-999	MEN-3K	COVERAGE	SST-2 ACCURACY
A	+0.029	+0.045	+0.118	50.1%	0.761
B	-0.036	+0.069	-0.010	50.1%	0.729
C	+0.700	+0.442	+0.771	100.0%	0.798
D	+0.585	+0.405	+0.702	50.1%	0.792
E	+0.005	+0.033	+0.096	50.1%	0.744

A, B and E have no measurable lexical similarity structure at all. B is slightly negative on two of three benchmarks, which is a correlation indistinguishable from none. Those same three models score 0.729 to 0.761 on sentiment, within 0.069 of GoogleNews. Mean-pooled logistic regression needs only that sentiment-bearing words occupy consistent directions. It does not need tiger to sit near cat. A report that evaluated downstream accuracy alone would conclude these embeddings are nearly as good as GoogleNews, and it would be wrong.

It also reframes what the warm start bought. D scores 0.585 on WordSim-353 against 0.029 for A, trained from scratch on the identical corpus with identical settings. The pretrained initialisation is not a 0.031-accuracy trick. It is the entire source of lexical structure in Model D.

MODEL D

What fine-tuning means for word2vec

Not what it means for a transformer. There are no frozen layers and no adapter. Initialise w_in from the published vectors, then run ordinary skip-gram training on movie reviews at a low learning rate.

12,032

rows seeded from GoogleNews

2,277

rows Google's vocabulary lacks

0.961

mean cosine, before against after

+0.031

accuracy over training from scratch

MOVED MOST

kidman		0.553
philippe		0.599
schwarzenegger		0.608
holm		0.647
griffiths		0.663
sheridan		0.669
evelyn		0.677
clooney		0.700

DID NOT MOVE AT ALL

oozes		1.000
priggish		1.000
plod		1.000
worshipful		1.000
drudgery		1.000
abysmally		1.000
poo		1.000
execrable		1.000

The drift ranking is a readout of corpus frequency. Everything at the top is an actor surname: frequent in reviews, rare in news, so heavily retrained. Everything at the bottom sits at cosine 1.000, unmoved: rare sentiment adjectives that appear once or twice in SST and never accumulate enough gradient. This answers "what did fine-tuning do" better than any single-word anecdote could.

01 Build the vocabulary from our corpus first

`intersect_word2vec_format` only overwrites rows that already exist. It does not import Google's three million words, which is what we want, but it needs stating.

02 Set `vectors_lockf` to ones explicitly

gensim 4.x does not create it. Without it the import either fails or silently freezes every row, producing a model that looks fine-tuned and is not.

03 Keep the learning rate low: alpha 0.005, 8 epochs

Google never published their output matrix, so `W_out` starts random and the first updates are partly spent re-learning a decoder. A normal learning rate scrambles the imported vectors in the first epoch.

04 Measure drift rather than assuming it

Cosine between each word's warm-start vector and its final vector. Mean 0.9609, median 0.9731, so the low learning rate held.

MODEL E, BEYOND THE REQUIREMENT

Writing skip-gram with negative sampling

The handout suggests taking a superficial look at the code. Writing it instead is the difference between having used word2vec and understanding it. `src/sngs.py` implements the vocabulary and counts, subsampling of frequent words, the unigram^{0.75} noise table, a dynamic window, the two embedding matrices, and linear learning rate decay.

`to_keyedvectors` then wraps `W_in` in a gensim object so E runs through the existing evaluation code unchanged.

	A, GENSIM	E, FROM SCRATCH
SST-2 accuracy	0.7611	0.7441
Wall clock	11.8 s	160.2 s
Words per second	537,887	39,571
Loss, first to last epoch	not exposed	4.2288 to 2.8681

The first version did not learn, and the failure is worth keeping. Loss sat at exactly 1.3863 for all ten epochs and accuracy came out 0.4992, which is chance on a balanced split. 1.3863 is $2 \cdot \ln 2$, the loss of a model whose every logit is zero. The cause was the loss reduction, not the sampling: calling `BCEWithLogitsLoss` once on the positives and once on the negatives averages over B entries and over B·k entries, so at a batch of 8,192 the effective per-pair learning rate was about $3e-6$. Measured gradient norm on `W_in` was exactly 0.0. Writing Mikolov's equation 4 directly, summing over the k negatives and averaging only over the batch, fixed it.

Two component checks were run before training anything, and they are what made that failure quick to localise. Subsampling keeps 17.6% of occurrences of `the`. The noise table maps `the` from a raw frequency of 4.30% down to a sampled 1.31% while lifting `execrable` from effectively zero to $2e-5$. Both are the 0.75 exponent behaving correctly.

DELIVERABLE 4

What these models cost to run

Query latency is brute-force `most_similar`, a dense matrix-vector product over the whole vocabulary, measured over 200 calls. Memory is an RSS delta measured in an isolated subprocess, against the size of the matrices actually retained.

MODEL	VOCAB	P50 MS	P95 MS	KEEPS	PREDICTED MB	MEASURED MB	RATIO
A	14,309	0.41	2.41	2	34.3	35.0	1.02
B	14,309	0.36	1.40	2	34.3	48.0	1.40
C	500,000	15.08	26.94	1	600.0	195.6	0.33
D	14,309	0.41	3.07	2	34.3	34.8	1.01
E	14,309	0.37	1.51	1	17.2	17.8	1.03

Query latency is linear in vocabulary, as brute force should be. C carries 35 times the vocabulary of the SST models and is 37 times slower at p50. That is the argument for an approximate index stated as a measurement: at 14,309 words nobody needs one, at 500,000 it is 15 ms per query, and at Google's full three million it would be roughly 90 ms.

The $2 \cdot V \cdot d \cdot 4$ prediction holds within 3% for A, D and E. B comes in at 1.40 because hierarchical softmax also keeps the Huffman tree's code and point arrays, which the formula does not count.

C does not fit, and the cause is not established. 500,000 by 300 float32 is 600 MB and the measured delta is 195.6 MB, a third of it. The likely explanation is macOS memory compression, which compresses inactive anonymous pages so `ru_maxrss` reports below the allocation. That is testable: the same measurement on a Linux node should come out near 600 MB. Until it runs, this is an open discrepancy and not a finding.

One more measurement decided how E trains. On this vocabulary, MPS is slower than CPU (64.3 against 44.6 ms per batch) and sparse gradients are slower than dense (52.6 against 44.6). The GPU loses because the model is small enough that launch and transfer overhead dominate. Sparsity loses for a specific reason: a batch of 8,192 pairs with 10 negatives touches roughly 90,000 index slots over 14,309 rows, so nearly every row updates on every step and there is no sparsity left to exploit. Both results should reverse on a larger corpus, which is the measurement text8 exists to take.

DELIVERABLE 2C

Where every model fails, and why it is the same place

MODEL	ERRORS / 1,821	NEGATION	CONTRAST	OTHER
C	368	102	64	202
D	379	96	64	219
A	435	101	72	262
E	466	114	79	273
B	493	120	62	311

Negation and contrast are 38 to 45% of every model's errors, against roughly 20% of the test set overall. That holds for GoogleNews too, which means it is not an embedding-quality problem. It is the pooling step: averaging discards word order, so no amount of embedding quality recovers a scope-of-negation judgement.

DELIVERABLE 2C, STATED EXPLICITLY FOR MODEL D

The handout asks for at least two test examples the model classifies correctly and at least two it gets wrong, for a named model. Model D, the fine-tuned one, is used here. All four are drawn from the same 1,821-sentence test split and are the highest-confidence cases in each category.

OUTCOME	GOLD	PREDICTED	CONFIDENCE	SENTENCE
correct	positive	positive	0.999	an intelligent , moving and invigorating film .
correct	positive	positive	0.998	a smart , sweet and playful romantic comedy .
incorrect	negative	positive	0.952	it 's a great deal of sizzle and very little steak .
incorrect	negative	positive	0.938	a well acted and well intentioned snoozer .

CONFIDENT FAILURES ACROSS MODELS

MODEL	SENTENCE	GOLD	PREDICTED	CONFIDENCE	CATEGORY
A	not a bad journey at all .	pos	neg	0.958	negation
A	an ultra-low-budget indie debut that smacks more of good intentions than talent .	neg	pos	0.917	other
A	a well acted and well intentioned snoozer .	neg	pos	0.898	other
D	never -lr- sinks -rr- into exploitation .	pos	neg	0.990	negation
D	there is n't a weak or careless performance amongst them .	pos	neg	0.973	negation
D	despite the long running time , the pace never feels slack -- there 's no scene that scr	pos	neg	0.955	negation

Each has a mechanical explanation. `not a bad journey at all` averages `not` and `bad` into negative territory. `there is n't a weak or careless performance amongst them` pools two strongly negative adjectives that the negation was supposed to invert. `a great deal of sizzle and very little steak` is an idiom whose negative reading lives entirely in the contrast between its clauses. The handout warns that a test set producing no failures must be made harder. That did not arise, since SST supplies these unaided.

DELIVERABLE 6

Installation, running and measurement

Four things cost real time, and each reported something other than its actual cause. They are recorded here because the handout asks for the experience, and because each has a specific tell.

01 gensim has no wheels for Python 3.14

The machine's default interpreter is 3.14. Installing gensim there tries to compile Cython from source and fails a long way in. Homebrew's Python 3.11 is what the virtual environment uses.

02 `import gensim` fails on a fresh unpinned install

gensim 4.3.x imports `scipy.linalg.triu`, which scipy 1.13 removed. The ImportError names gensim, so it reads as a broken gensim rather than a version conflict. `requirements.txt` pins `scipy<1.13` and `numpy<2`.

03 The SST-2 split with hidden labels

The GLUE copy sets every test label to -1. Accuracy against it is meaningless and looks plausible. `src/data.py` asserts the test split is two-class and raises otherwise.

04 Whole-process peak RSS is not a model measurement

The first memory table compared process peak RSS against the matrix prediction and reported 297.8 MB for a model whose matrices are 34.3 MB. Peak RSS includes the interpreter, gensim, numpy, torch and the resident corpus. `src/measure_mem.py` now measures an RSS delta in an isolated subprocess per model.

Reproducing everything on this page is four commands after `pip install -r requirements.txt`: `python -m src.main` trains A and B and loads C, `python -m src.run_eval` adds the similarity, latency and error measurements, `python -m src.measure_mem` fills the memory table, and `python -m src.build_report` regenerates this page. Model E is `python -m src.run_e` and takes 160 seconds on CPU.

PROVENANCE

Build log

47fa014	HW2 phases 0-3 env, SST-2 data, gensim models A and B, sentiment pipeline	done
5cb380c	Phase 4 phrase-level SST corpus (4.7x text), Model C GoogleNews-300, OOV diagnostic	done
c86ba65	Phase 5 Model D, GoogleNews warm start fine-tuned on SST, with cosine drift analysis	done
340fe91	Phase 10 Model E, skip-gram with negative sampling from scratch in PyTorch	done
5551046	Phases 6-8 similarity benchmarks, latency and memory measurement, error mining	done

f8e08a2	Phase 9 generated HTML report in docs/, architecture SVGs, GitHub Pages layout	done
30690e9	Deliverable 3 five ranked test examples with retrieval; foreground sentiment in report	done
76a6f4a	Deliverables 2b and 6; fix misleading cosine field name; build PDF and submission zip Deliverables 2b and 6; fix misleading cosine field name; build PDF and submission zip	done
454d16c	Match HW1 deploy structure Vercel on public/, drop GitHub Pages, untrack instructor handout	done
—	text8 Train A, B and E on 17M tokens; retest the MPS and sparse crossovers	queued
—	Hogwild scaling words/sec against worker count on a multi-core node	queued
—	Hyperparameter sweep training cost against accuracy for window, dimension and negatives	queued

Every figure on this page is read from `results/*.json` at build time by `src/build_report.py`. Nothing is typed into the prose, so re-running a model and rebuilding cannot leave a stale number behind.